# TEMA 9 – BBDD NO RELACIONALES. INTRODUCCION A MONGODB.

## 1. Que es una estructura NOSQL

Son estructuras que nos permiten almacenar información cuando las Bases de Datos Relacionales nos dan problemas de ESCALABILIDAD y RENDIMIENTO donde hay muchos usuarios concurrentes y muchas consultas diarias.

**Sus características principales son:**

- Cada objeto se almacena en un documento.
- Los grupos de objetos se almacenan en colecciones.

**Ejemplo de documento**

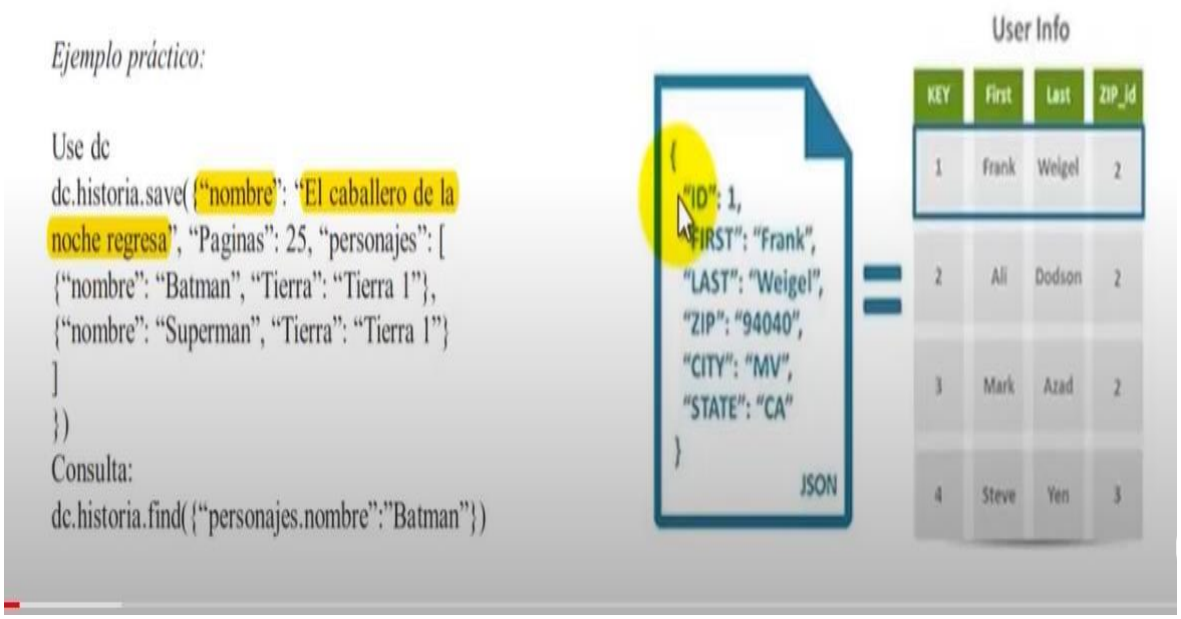
```
Documento = {
Nombre: "Luís",
Apellidos: "Martinez",
Edad: 18,
 Aficiones: ["fútbol","senderismo","tenis"],
 Amigos: [
  {
   Nombre:"Monica",
   Edad:20
  },
  {
   Nombre:"Andrés",
   Edad:24
  }
 ]
}
```

# 2. Que es una Base de Datos Documental

Una Base de Datos contiene un **conjunto de colecciones.**

Una **colección es un conjunto de documentos**, que se pueden relacionar con las tablas del modelo relacional, pero ahora en NOSQL podemos guardar documentos con distintos atributos.

Un **documento es un conjunto de campo**, donde un campo es una pareja formada por una clave o llave y un valor donde la clave es el nombre del campo y el valor es su contenido.




**Las Bases de Datos Documentales son aquéllas que guardan estructuras de datos en documentos JSON (JavaScript Object Notation**). También se usan estructuras XML para definir los documentos.

**JSON es un formato de datos basado en la sintaxis de JavaScripts** (literales de matrices y de objetos).

# 3. Fundamentos

**MongoDB: es un almacén NoSQL de tipo documental**

**Su almacenamiento no está estructurado:**

- Cada objeto se almacena en un documento
- Los grupos de objetos se organizan en colecciones de documentos

**Al ser no estructurado carece de diseño previo:**

- La colección no se define: sus documentos pueden tener cualquier conjunto de campos e incluso el mismo nombre de campo puede contener distintos tipos de datos
- El esquema global (diseño de BD) no se define. No existe un LDD
- La colección se crea al añadir el primer elemento en ella

**MongoDB utiliza para el almacenamiento y la transferencia de información el formato BSO**N (Binary JSON)

- ✓ JSON (JavaScript Object Notation) es un formato de intercambio de datos: Ligero, simple y eficiente para grandes volúmenes de datos yEs un subconjunto del lenguaje Javascript
- ✓ BSON es una versión serializable binaria de JSON

**Un fichero en BSON** puede contener la definición de uno o más objetos o documentos**. Un objeto BSON** es una lista ordenada de elementos agrupados por llaves {} y separados por comas. Cada elemento es una tripleta <nombre, tipo, valor>

**Tipos en BSON (https://docs.mongodb.com/manual/reference/bson-types/)**

- Tipos simples: entero, real (IEEE754), string, booleano, null, fecha.
- Tipos especiales: expresiones regulares, códigos JavaScripts
- Tipos compuestos:
  - ✓ Otro objeto embebido: la definición de otro documento distinto referenciándolo por su nombre
  - ✓ Un array de objetos: utilizando corchetes []

# 4. Identificadores

**Su tipo es Object Id y es un string de 12 bytes** presentado normalmente en formato hexadecimal.

**Es un identificador único**, ordenado y fácil de crear.

**Se compone de tres campos:**

- ✓ Un TIMESTAMP de  bytes que representa la fecha del objeto en segundos
- ✓ Un campo único de 5 bytes
- ✓ Un contador de 3 bytes autoincrementado.

# 5. Manejo de Bases de Datos y Colecciones.

| ORDEN | Descripción |
|---|---|
| db | Muestra la BD activa (*test* al entrar en sesión) |
| show dbs | Muestra las BD's disponibles |
| use nombre_de_bd | Pasa a utilizar la BD indicada. Si no existe la crea automáticamente al referenciarla por primera vez aunque está todavía vacía |
| show collections | Muestra las colecciones de la base de datos con la que se está trabajando |
| db.nombre_col.insert (documento) | Inserta un documento en una determinada colección de la BD en uso. Si la colección no existe, se crea automáticamente |
| db.createCollection() | Crea una colección con parámetros de configuración específicos |
| db.nombre_col.count() | Devuelve el número de elementos de la colección |

# 6. Operaciones CRUD

| Creación | Descripción |
|---|---|
| db.nombre_col.insert (var_name) | [C] Inserta un documento en una determinada colección de la BD en uso utilizando una variable |
| db.nombre_col.insertOne(doc) | [C] Inserta un documento en una determinada colección de la BD definiendo el documento directamente en la invocación |
| db.nombre_col.insertMany ([doc1, doc2, ...]) | [C] Inserta un array de documentos en una determinada colección de la BD |
| db.nombre_col.find (filter, projection) | [R] Lectura de un conjunto de campos (projection) utilizando un filtro con condiciones |
| db.nombre_col.save(doc) | [U] Actualiza un determinado documento y lo crea si no existe |
| db.nombre_col.remove (filter, just_one) | [D] Borra los documentos que cumplen con el filtro |
| db.nombre_col.remove () | [D] Borra todos los documentos de una colección |
| db.nombre_col.updateOne ({filter}, {$set:{modificación}}) | [U] Actualiza los documentos que cumplen las condiciones de filtrado |
| db.nombre_col.updateOne ({filter}, {$unset:{borrado}}) | [U] Borra los documentos que cumplen las condiciones de filtrado. |
| db.nombre_col.updateMany ({filter}, {$set:{...}}) | [U] Actualiza todos los documentos que cumplen las condiciones de filtrado |

# 7. Búsquedas

**La función find() admite dos parámetros: filtro y proyección**

**La expresión vacía {} utilizada como filtro** indica una selección completa

**Los filtros son condiciones de restricción que se definen sobre los campos del documento y tiene la forma campo: {op: valor}** . Se pueden enumerar tantas como se deseen separándolas por comas

{precio:{$gt:100, $lt:200}}  indica la expresión 100 < precio < 200

Si se indican varias condiciones, se toma la última

{precio:{$gt:100}, {$lt:500}}  indica la expresión precio < 500

**Las condiciones se pueden agrupar mediante conectores:**

{$and: [{exp1}, {exp2},...]}

{$or: [{exp1}, {exp2},...]}

{$not: {exp}}

{$nor: [{exp1}, {exp2},...]}

# 8. Operadores

| Name | Description |
|---|---|
| $eq | Matches value that are equal to a specified value |
| $gt, $gte | Matches values that are greater than (or equal to a specified value |
| $lt, $lte | Matches values less than or ( equal to ) a specified value |
| $ne | Matches values that are not equal to a specified value |
| $in | Matches any of the values specified in an array |
| $nin | Matches none of the values specified in an array |
| $or | Joins query clauses with a logical OR returns all |
| $and | Join query clauses with a loginal AND |
| $not | Inverts the effect of a query expression |
| $nor | Join query clauses with a logical NOR |
| $exists | Matches documents that have a specified field |

# 9. Filtros sobre Arrays.

**Si una restricción compuesta por varias condiciones se define sobre un campo que es un array**, la restricción se hará true si cada una de las condiciones es cumplida por al menos un elemento del array

**Si lo que se desea es seleccionar un elemento del array que cumple un determinado conjunto de restricciones se utiliza el operado $elemMatch**

db.collection.find( { campo_array: {$elemMatch: {exp1, exp2, …}}} )

**Si se desea recuperar los documentos con un campo de tipo array de cierto tamaño se utiliza el operador $size**

db.collection.find( { campo_array: {$size: 10}} )

# 10. Proyecciones.

**Una proyección es una secuencia de inclusiones (campo:1) o de exclusiones (campo:0) separada por comas y encerradas entre {}**

db.libros.find( {}, {Título:1} )

db.libros.find( {Apellido:“Asimov"}, {Apellido:0, Título:0} )

**El identificador del documento (_id) se incluye por defecto en todas las búsquedas aunque se puede excluir**

db.libros.find( {}, {Nombre:1, _id:0} )

**Al proyectar campos que son de tipo array, se puede controlar mediante el operador $slice** el número de elementos que se muestran

# 11. Agregaciones (equivale al GROUPBY y HAVING de SQL).

- La agregación permite agrupar y procesar los documentos de una colección

- Puede ser de tres tipos:

1. **Simple.**
   - Los métodos count() y distinct() seleccionan documentos de una colección y los cuentan o extraen los valores únicos

     db.libros.distinct( 'Title' )

   - Existe una forma genérica utilizando group() indicando el subconjunto a procesar y proporcionando una función de procesado
2. **Pipeline**
   - Es un framework que bajo el modelo de un pipeline de varias fases puede transformar los documentos y calcular funciones agregadas
3. **Map-reduce**
   - Es un paradigma de computación que condensa grandes volúmenes de información y calcula de forma distribuida funciones agregadas

## Operadores de **fase**

- **$match:{query}** : filtra la colección aplicando la selección dada
- **$limit:num** : trunca la colección al tamaño dada (n primeros documentos)
- **$skip:num** : omite una cantidad dada de documentos (los n primeros)
- **$project:{spec}** : proyecta campos de los documentos (mismo funcionamiento que la proyección del método find() )
- **$facet**: procesa diferentes pipelines en la misma fase. Es útil para análisis multidimensionales
- **$redact:{expr}** : permite truncar documentos (prune) o subdocumentos (descend), o mantenerlos todos (keep), acorde con condiciones sobre el documento.
- **$sort:{spec}** : ordena los documentos siguiendo el criterio dado
- **$out:"col_name"** : crea y almacena una colección con los docs. Recibidos
- **$unwind:array** : sobre un elemento de tipo array, crea un documento para cada valor del array
- **$group:{query}** : agrupa la colección (crea colección agrupada)

- Operadores de **agrupación** para **$group**
  - $sum : sumatorio de los valores del grupo (un valor por documento)
  - $sum : media de los valores del grupo (un valor por documento)
  - $max : máximo de los valores del grupo
  - $min : mínimo de los valores del grupo
  - $first : valor del primer documento del grupo
  - $last : valor del último documento del grupo
  - $push : array con todos los valores del grupo (un valor por documento)
  - $addToSet : como el anterior, pero descartando valores repetidos

- Operadores de expresión para añadir resultados
  - Aritméticos: suma ($add), resta ($subtract), producto ($multiply), división ($divide), residuo ($mod)
  - Sobre Strings: concatenación ($concat), substring ($substr), minúsculas ($toLower), mayúsculas ($toUpper), comparación ($strcasecmp)
  - Comparación y lógicos: $and, $or, $not, $eq, $ne, $gt, $lt,...
  - Dede Conjuntos: equivalencia ($setEquals), inclusión ($setIsSubset), unión ($setUnion), intersección ($setIntersection), diferencia ($setDifference)

# EJEMPLO DE AGREGATE

**Se usa para realizar operaciones avanzadas y permite realizar cálculos, filtrados y agrupaciones.**

- $match: Filtra los documentos según ciertos criterios, similar a una cláusula WHERE en SQL.
- $group: Agrupa los documentos por un campo específico y permite realizar operaciones de agregación, como contar, sumar o calcular promedios.
- $sort: Ordena los documentos según uno o más campos.
- $project: Modifica la forma de los documentos, permitiendo incluir o excluir campos específicos y crear nuevos campos.
- $limit: Limita el número de documentos devueltos.
- $skip: Omite un número específico de documentos en la salida.

**Ejemplo: Sacar los documentos de ciclos (ordenado por el total de alumnos en orden descendente) incluyendo solo aquellos del grupo 1, agrupando los resultados por el nombre del ciclo y sumando el número de alumnos, y finalmente**

```
db.ciclos.aggregate([
  { $match: { Grupo: 1 } }, // Filtra solo los grupos 1
  { $group: { _id: "$Nombre", totalAlumnos: { $sum: "$NumAlumnos" } }
}, // Agrupa por Nombre y suma NumAlumnos
  { $sort: { totalAlumnos: -1 } } // Ordena por totalAlumnos de forma descendente
                                    ]);
```

```
{
  _id: 'ASIR',
  totalAlumnos: 2
}
{
  _id: 'DAW',
  totalAlumnos: 2
}
{
  _id: 'DAM',
  totalAlumnos: 2
}
```

# 12. Manejo de dos colecciones. Consultas. Uso de Lookup.

En una Base de datos NO relacional, de algún modo hay que indicar que campo de una colección se relaciona con otro. En MongoDB esta $LOOKUP que es una etapa del PIPELINE DE AGREGACION que permite combinar documentos de dos colecciones diferentes.

Su función es similar a un JOIN en Bases de Datos Relacionales. El resultado siempre es un array con los documentos que cumplen la condición de la relación.

**Sintaxis:**

```
{
    $lookup: {
    from: <coleccion_a_unir>,
    localField: <campo_de_la_coleccion_actual>,
    foreignField: <campo_de_la_coleccion_externa>,
    as: <nombre_del_nuevo_campo>
}
}
```

**Ejemplo:** Mostrar todos los documentos de la colección ciclos, pero añadiendo un nuevo campo llamado profesor que contiene un array con los documentos de los profesores relacionados.

```
db.ciclos.aggregate([
{
    $lookup: {
    from: "profesores",
     localField: "profesores",
    foreignField: "_id",
    as: "profesor"
    }
}
])
```

**EVIDENTEMENTE, SI HAY QUE RELACIONAR 3 COLECCIONES PUES HAY QUE PONER DOS LOOKUP, Y ASI SUCESIVAMENTE…**

# 13. Errores de Compilación.

**En MongoDB, cuando ejecutas una consulta en el shell y ocurre un error, la información sobre el error se muestra directamente en la consola. Generalmente, verás un mensaje que incluye:**

1. Tipo de error: Una descripción del tipo de error (por ejemplo, `MongoError`, `SyntaxError`, etc.).
2. Mensaje de error: Una descripción más detallada del problema.
3. Ubicación: No se muestra la línea exacta pero se puede obtener pistas sobre qué parte de la consulta es problemática.

**Ejemplo de salida de error**

Cuando hay un error en una consulta, la salida podría parecerse a esto:

MongoError: E11000 duplicate key error collection: test.collection index: _id_ dup key: { : "some_id" }

**Algunas soluciones que podemos revisar:**

1. Errores de sintaxis: Revisa la consulta para asegurarte de que la sintaxis sea correcta.
2. Errores de tipo: Asegúrate de que los tipos de datos en la consulta coincidan con los tipos esperados en la base de datos.
3. Índices duplicados: Si ves un error de clave duplicada, verifica que no estés intentando insertar un documento con un `_id` que ya existe.

Se puede usar **db.getLastError()** después de una operación de escritura para obtener más detalles sobre el último error.

## TIPOS DE ERRORES

### 1. Errores de Sintaxis

- Descripción: Ocurren cuando la consulta no sigue la sintaxis correcta de MongoDB.

- Ejemplo: Falta de comillas, paréntesis desbalanceados, etc.

- Mensaje típico: `SyntaxError: unexpected token`

### 2. Errores de Conexión

- Descripción: Surgen cuando no se puede establecer una conexión con la base de datos.

- Ejemplo: Problemas de red, el servidor no está en funcionamiento.

- Mensaje típico: `MongoNetworkError: failed to connect to server`

### 3. Errores de Autenticación

- Descripción: Ocurren si las credenciales proporcionadas son incorrectas.

- Ejemplo: Usuario o contraseña erróneos.

- Mensaje típico: `MongoError: Authentication failed`

### 4. Errores de Escritura

- Descripción: Suceden al intentar realizar operaciones de escritura que no son válidas.

- Ejemplo: Insertar un documento con un `_id` que ya existe.

- Mensaje típico: `MongoError: E11000 duplicate key error`

**5. Errores de Tipo**

- Descripción: Ocurren cuando hay un conflicto de tipo de datos.
- Ejemplo: Intentar comparar un número con una cadena.
- Mensaje típico: `MongoError: Invalid BSON type`

**6. Errores de Consulta**

- Descripción: Suceden cuando la consulta no puede ser ejecutada correctamente.
- Ejemplo: Utilizar un campo que no existe en el documento.
- Mensaje típico: `MongoError: Field 'x' not found`

**7. Errores de Agregación**

- Descripción: Se producen durante el uso de pipelines de agregación.
- Ejemplo: Utilizar un operador no válido o una etapa mal definida.
- Mensaje típico: `MongoError: Unrecognized pipeline stage name`

**8. Errores de Indexación**

- Descripción: Ocurren cuando hay problemas al crear o utilizar índices.
- Ejemplo: Intentar crear un índice en un campo que no existe.
- Mensaje típico: `MongoError: Cannot create index on field 'x'`

**9. Errores de Límites**

- Descripción: Se producen cuando se superan los límites de tamaño de documentos o resultados.
- Ejemplo: Intentar insertar un documento que exceda el tamaño máximo.
- Mensaje típico: `MongoError: Document exceeds maximum size`

## EJEMPLOS DE ERRRORES

```
> Comprobamos db.amigos.find({Nombre: "Jose Carlos " })
✖ ▸ SyntaxError: Missing semicolon. (1:11)

  > 1 | Comprobamos db.amigos.find({Nombre: "Jose Carlos " })
    |             ^
```